

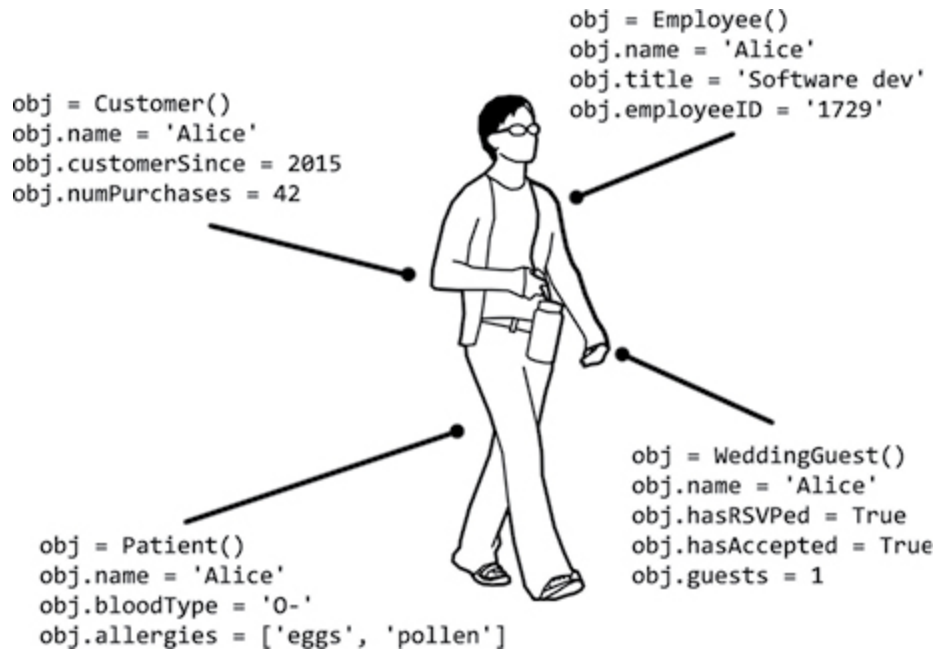


Figure 15-3: Four objects made from different classes that represent the same real-world person, depending on what the software application needs to know about the person

Also, the information contained in your classes should depend on your program's needs. Many OOP tutorials use a `car` class as their basic example without noting that what goes into a class depends entirely on the kind of software you're writing. There's no such thing as a generic `car` class that would *obviously* have a `honkHorn()` method or a `numberOfCupholders` attribute just because those are characteristics real-world cars have. Your program might be for a car dealership web app, a car racing video game, or a road traffic simulation. The car dealership web app's `car` class might have `milesPerGallon` OR `manufacturersSuggestedRetailPrice` attributes (just as a car dealership's spreadsheets might use these as columns). But the video game and road traffic simulation wouldn't have these attributes, because this information isn't relevant to them. The video game's `car` class might have an `explodeWithLargeFireball()` method, but the car dealership and traffic simulation, hopefully, would not.

Creating Objects from Classes

You've already used classes and objects in Python, even if you haven't created classes yourself. Consider the `datetime` module, which contains a